

Lab#11(Procedure)

Task01:

Create Products Table:

```
• CREATE TABLE Products (  
    ProductID INT PRIMARY KEY AUTO_INCREMENT,  
    ProductName VARCHAR(100),  
    Category VARCHAR(50),  
    Price DECIMAL(10,2),  
    Quantity INT  
);
```

Create Add Product Stored Procedure

```
DELIMITER $$  
CREATE PROCEDURE AddProduct(  
    IN ProductName VARCHAR(100),  
    IN Category VARCHAR(50),  
    IN Price DECIMAL(10,2),  
    IN Quantity INT  
)  
BEGIN  
    INSERT INTO Products (ProductName, Category, Price, Quantity)  
    VALUES (ProductName, Category, Price, Quantity);  
END $$  
DELIMITER ;
```

Execution Example

```
CALL AddProduct('Laptop', 'Electronics', 75000.00, 10);  
call AddProduct ('Phone', 'Electronics', 30000.00, 20);
```

Create GetProductByID Stored Procedure

```
DELIMITER $$  
CREATE PROCEDURE GetProductByID(  
    IN ProductID INT  
)  
BEGIN  
    SELECT * FROM Products  
    WHERE ProductID = ProductID;  
END $$  
DELIMITER ;
```

Execution Example:

```
36 CALL GetProductByID(1);  
37 #Create GetAllProducts Stored Procedure
```

Result Grid |  Filter Rows: | Export:  | Wrap

	ProductID	ProductName	Category	Price	Quantity
1	1	Laptop	Electronics	75000.00	10
2	2	Phone	Electronics	30000.00	20
3	3	Laptop	Electronics	75000.00	10
4	4	Phone	Electronics	30000.00	20

Create GetAllProducts Stored Procedure

```
DELIMITER $$  
CREATE PROCEDURE GetAllProducts()  
> BEGIN  
    SELECT * FROM Products;  
- END $$  
DELIMITER ;
```

Execution Example:

```
6 CALL GetAllProducts();  
7
```

Result Grid |  Filter Rows: | Export:  | Wrap

	ProductID	ProductName	Category	Price	Quantity
1	1	Laptop	Electronics	75000.00	10
2	2	Phone	Electronics	30000.00	20
3	3	Laptop	Electronics	75000.00	10
4	4	Phone	Electronics	30000.00	20

Task 02:

Create Students Table

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY auto_increment,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    DateOfBirth DATE,  
    Grade VARCHAR(10)  
);
```

Create Add Student Stored Procedure

```
DELIMITER $$  
CREATE PROCEDURE AddStudent(  
    IN FirstName VARCHAR(50),  
    IN LastName VARCHAR(50),  
    IN DateOfBirth DATE,  
    IN Grade VARCHAR(10)  
)  
BEGIN  
    INSERT INTO Students (FirstName, LastName, DateOfBirth, Grade)  
    VALUES (FirstName, LastName, DateOfBirth, Grade);  
END $$  
DELIMITER ;
```

Execution Example:

```
call AddStudent ('Ali', 'Khan', '2005-06-15', '10th');  
call AddStudent ('Sara', 'Ahmed', '2006-09-20', '9th');  
#Create UpdateStudent Stored Procedure
```

Create Update Student Stored Procedure

```
DELIMITER $$  
CREATE PROCEDURE UpdateStudent(  
    IN p_StudentID INT,  
    IN p_FirstName VARCHAR(50),  
    IN p_LastName VARCHAR(50),  
    IN p_DateOfBirth DATE,  
    IN p_Grade VARCHAR(10)  
)  
BEGIN  
    UPDATE Students  
    SET  
        FirstName = IFNULL(p_FirstName, FirstName),  
        LastName = IFNULL(p_LastName, LastName),  
        DateOfBirth = IFNULL(p_DateOfBirth, DateOfBirth),  
        Grade = IFNULL(p_Grade, Grade)  
    WHERE StudentID = p_StudentID;  
END $$  
DELIMITER ;
```

Execution Example:

```
CALL UpdateStudent(1, 'John', 'Smith', '2004-03-15', 'A');
```

Create Get_All_Students Stored Procedure

```
DELIMITER $$  
CREATE PROCEDURE GetAllStudents()  
BEGIN  
    SELECT * FROM Students;  
END $$  
DELIMITER ;
```

Execution Example:

```
CALL GetAllStudents();
```

Delete Student Procedure

```
DELIMITER $$
CREATE PROCEDURE DeleteStudent(
    IN p_StudentID INT
)
BEGIN
    DELETE FROM Students
    WHERE StudentID = p_StudentID;
END $$
DELIMITER ;
#execute
CALL DeleteStudent(1);
```

Task 03:

Create tables:

```
#student table
CREATE TABLE Student01 (
    StudentID INT AUTO_INCREMENT PRIMARY KEY,
    StudentName VARCHAR(100),
    Department VARCHAR(50)
);
#result
CREATE TABLE Results (
    ResultID INT AUTO_INCREMENT PRIMARY KEY,
    StudentID INT,
    CourseName VARCHAR(50),
    Marks INT,
    Grade CHAR(2),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
```

Insert Sample Students

```
INSERT INTO Student01 (StudentName, Department)
VALUES
('Ali Khan', 'Computer Science'),
('Sara Ahmed', 'Software Engineering'),
('Hassan Raza', 'Data Science');
```

Create Stored Procedure

```
DELIMITER //  
  
CREATE PROCEDURE AddStudentResult (  
    IN p_StudentID INT,  
    IN p_CourseName VARCHAR(50),  
    IN p_Marks INT  
)  
  
BEGIN  
    DECLARE v_Grade CHAR(2);  
  
    -- Grade calculation logic  
    IF p_Marks >= 85 THEN  
  
        -- Insert data into Results table  
        INSERT INTO Results (StudentID, CourseName, Marks, Grade)  
        VALUES (p_StudentID, p_CourseName, p_Marks, v_Grade);  
    END //  
DELIMITER ;
```

Execute (Call) the Procedure

```
CALL AddStudentResult(1, 'Database Systems', 92);  
CALL AddStudentResult(2, 'Operating Systems', 76);  
CALL AddStudentResult(3, 'Discrete Structures', 59);  
CALL AddStudentResult(1, 'Data Structures', 35);
```

Display the Data

```
SELECT  
    S.StudentName,  
    S.Department,  
    R.CourseName,  
    R.Marks,  
    R.Grade  
FROM Results R  
JOIN Students S ON R.StudentID = S.StudentID;
```